Creative Media Analyzer

Roadmap

Current state, build journey, and what's next

Author Theo Rajan

Date 2026-05-13

Deployment https://media-analyzer-theta.vercel.app

Repository github.com/theodoreheroldrajan-sketch/media-analyzer

1. Current state

Deployed at `https://media-analyzer-theta.vercel.app`. The root URL redirects to `/demo` to keep portfolio visitors out of the production app's API budget. Real-app pages remain reachable by direct URL (`/setup`, `/dashboard`, and the other wizard steps) for the operator only.

Architecture.

- Next.js 16.2.6 (App Router, Turbopack) on Vercel Hobby.
- TypeScript 5, React 19.2.4, Tailwind 4.
- Supabase Postgres (eu-west-2) with nine tables and a creatives storage bucket. Database types generated from the schema and committed.
- Claude Haiku 4.5 (model id `claude-haiku-4-5-20241022`) via the Anthropic SDK, called with forced tool_use over Node.js runtime (Edge runtime is incompatible with the SDK).
- Streaming NDJSON for extraction progress, bypassing Vercel's 10-second function timeout.

Environment requirements: `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY` (and a service role key for server routes), and `ANTHROPIC_API_KEY`. All three are set in Vercel for production, preview, and development.

2. Build journey

Reconstructed from 50 commits across roughly three weeks. The journey, not the destination, is the point of this section.

2.1 Streamlit prototype (April 22-23)

Five commits. The bet was that Claude vision with tool_use could replace manual creative tagging entirely. The prototype was a single-page Streamlit app: upload images, hit an Anthropic call, render the extracted variables in a table. Done in two evenings.

What this proved. Forced tool_use produces consistent structured output across diverse images. Cost per image was small enough that batches of 50-100 were feasible.

What this exposed. Streamlit is the wrong shell for a multi-step workflow. The product needed a wizard, persistent state, and proper async progress. The Streamlit version is preserved in `legacy/` as a reference but is no longer the deployed surface.

2.2 Next.js shell and Supabase foundation (May 11)

Three PRs in one evening. Pivoted to Next.js 16 App Router with a sidebar-based nine-step stepper. Created the Supabase project, designed the nine-table schema with a snapshot model on `performance_rows` (an `is_latest` flag instead of destructive overwrites). Imported the design system from a Claude Design export.

The snapshot decision. Marketers re-export CSVs constantly. Hard-overwriting older performance rows would erase history and re-running mapping would be expensive. The snapshot model means each upload creates a new `performance_upload` row with its own batch of `performance_rows`; the previous batch's rows are flipped to `is_latest = false`. Cheap, reversible, audit-friendly.

2.3 Wizard steps 1-8 (May 11-12)

Eight PRs in roughly 24 hours, one per step. Setup form wiring, image plus CSV uploads, the six-method matching cascade, variable schema builder with category templates, AI extraction with streaming, dashboard with trust score and group-by analytics, settings with CSV export and project delete, polish (error boundary, sidebar completion tracking).

Notable inflection - Edge runtime fail. The extraction route was initially deployed on Vercel Edge for the streaming. It failed at deploy time: the Anthropic SDK imports node:fs and node:path which Edge does not provide. The fix was to drop back to Node.js runtime; the streaming approach (ReadableStream piping NDJSON chunks) works there too and is what shipped.

Notable inflection - lazy Supabase client. Supabase client instantiation at module top-level broke Next.js build because environment variables are not available at build time. Refactor to a `getSupabase()` function pattern made the build pass.

2.4 Interactive demo (May 13)

Two PRs in the morning. The first added a self-contained demo at `/demo` with 40 deterministically-generated fake creatives ("GlowLab", a fictional DTC skincare brand) and an enhanced, componentized dashboard. The second split the dashboard into Pro and Lite paths, added 120-creative Pro fake data, and built the five chart types (bar, scatter, regression scatter, distribution, heatmap) as inline SVG without any chart library.

Why mock the Pro stats. A real OLS backend can be implemented once, against real data, with empirical decisions about regularization and interactions. Synthesizing those decisions on fake data risked shipping a model that did not survive contact with real distributions. The demo shows the experience; the production code path stops at group-by for now.

Same-day instructions overhaul. The instructions page was rewritten to include step-by-step Meta Ads Manager and Google Ads export guides, sample CSV previews, common pitfalls (granularity, encoding, multi-row creatives), and a pre-upload checklist. This is the gatekeeping document - a portfolio visitor reading it understands the data preparation work involved and decides whether to

request a real demo.

2.5 Access control lockdown (May 13 evening)

Root `/` changed from a marketing landing page to a redirect to `/demo`. Real-app pages still resolve at their original URLs but are not discoverable from anywhere user-facing. This protects the operator's Supabase storage and Anthropic API budget from accidental usage by portfolio visitors. The trade-off is that the real app is now discoverable by URL guessing only; this is acceptable for a portfolio piece run as a single-operator tool.

3. What's done (production)

Concrete capabilities currently shipped:

- Image upload (PNG/JPEG, 10 MB cap) to Supabase Storage with thumbnail-grid preview.
- CSV parser with auto-detected column mapping; supports Meta Ads Manager and Google Ads exports out of the box.
- Six-method matching cascade with confidence scoring (exact, normalised, embedded platform ID, prefix, contains, fuzzy Levenshtein).
- Variable schema builder: 24 universal definitions plus 4-6 from any of 10 category templates, plus custom variables in Pro.
- AI extraction pipeline: Claude Haiku 4.5 with forced tool_use, streaming NDJSON over Node.js runtime, per-image cost tracking at the published Haiku rates.
- Dashboard: metric switcher (CTR/CPC/CPA/CVR/ROAS), trust score with six sub-score bars, variable explorer (bar chart in Lite, plus four more chart types in Pro), sortable variable performance table, ranked creative gallery, insights panel.
- Three CSV export types (variables-only, performance-only, combined).
- Project delete with cascade across nine tables and the storage bucket.
- Interactive demo at `/demo` with Pro/Lite mode chooser, 40 or 120 fake creatives, full Pro UI with mocked regression statistics.
- Comprehensive instructions page (Meta export, Google export, naming convention, sample CSVs, common pitfalls, pre-upload checklist).

4. What's planned next

Priority order. Timelines are deliberately vague ("post-v1") rather than dated, because each depends on access to a real 100-plus creative dataset.

- **Production OLS regression backend.** Move the mocked Pro statistics out of `demo-data.ts` and into a real server-side regression pipeline. Gate on first real 100-plus creative dataset to make empirical choices about weighting, interactions, and regularization.
- **AI insights narration.** Replace placeholder insights panel with Claude-generated explanations grounded in the actual coefficients and deltas. Requires the production regression to land first so the narration has real numbers to reference.
- **Personal knowledge-base layer.** Markdown notes (Obsidian-compatible) the operator maintains per brand. Notes are injected into the extraction prompt so repeated campaigns inherit context. Confirm with Theo - exact scope and storage model to be decided.
- **Volume-weighted confidence labels.** Replace the discrete n-threshold confidence label with a continuous score that incorporates impression volume so high-volume small-n groups are not penalized.
- **Multi-row creative aggregation.** Detect and aggregate when one creative appears in multiple CSV rows (same image, multiple ad sets) at parse time rather than relying on the operator to dedupe.
- **Additional platforms.** TikTok and LinkedIn CSV column mappings.

5. Limitations

Mandatory section. The honest list of what does not work yet.

- Static image creatives only. Video and dynamic creatives are not supported.
- Single-user. No auth, no shared workspaces, no comments.
- Vercel Hobby tier. Practical batch ceiling around 150-200 creatives per extraction run; above that, batching and resumable runs become necessary.
- CSV uploads only. No real-time Meta or Google API integration.
- Pro statistics in the dashboard are mocked. Real OLS backend is not yet built.
- Root URL redirects all visitors to the demo. Operator must bookmark real-app URLs for personal use.
- Demo data is fictional with deliberate correlations. The Pro mode regression table values, while internally consistent, are not the output of a real regression.
- Group-by analysis is descriptive only. No p-values, no confidence intervals on deltas, no multiple-comparison correction.
- No temporal handling. Date ranges from the CSV are read but seasonality is not modeled.
- Mapping ceiling. The fuzzy Levenshtein method handles minor filename variation but not creatives renamed entirely between export and upload.

6. Parked and future

Items deferred without a timeline. Whether they ship depends on real-world demand.

- Obsidian-style personal knowledge base for cross-campaign context.
- Video creative support (frame extraction, motion variables, audio).
- Real-time Meta/Google API integration (replaces CSV uploads).
- Mixed-effects models for campaign-nested data.
- Bayesian regression for small-sample uncertainty quantification.
- Multi-user collaboration and shared workspaces.
- Saved "views" - persisted dashboard filter and metric configurations.

7. Open strategic questions

Decisions not yet made. Listed so the document represents the project honestly rather than retrofitting a finished narrative.

- **Productize or keep as a personal tool.** The architecture supports either. Productization needs auth, billing, and a support story. Personal-tool use needs only the current shape.
- **Paid media versus social.** The shipped functionality is paid-ads-shaped. Organic social would need a different metric set (saves, shares, comment sentiment) and probably a different schema. Both are open.
- **Pricing model.** If productized: subscription tiered on Lite/Pro? Single price? Per-creative usage?
- **Lighthouse customer or open beta.** The Betterhalf.ai-style customer (high spend, technical team) is the lighthouse for Pro validation. The open question is whether to widen access before that validation is done, or after.

Appendix: Confirm with Theo

Items not determinable from the repository.

- Whether the deployed URL should be promoted publicly or kept for portfolio-share-only.
- Personal knowledge-base layer - intended scope and rough priority versus the production regression backend.
- Whether the Pro/Lite split should be carried over to a future subscription model, and what naming/pricing would look like.

- Whether to commit the next 100-plus creative dataset path (Betterhalf.ai or otherwise) as the explicit gating event for the regression backend, or leave it open.
- Whether the Streamlit `legacy/` folder should be removed before the next public-facing iteration or kept for the build-journey narrative.